TRABALHO FINAL SOBRE PADRÕES DE PROJETO[1]

1. Contexto e objetivo

Design patterns são soluções reutilizáveis para problemas recorrentes de projeto, mas dominá-los exige mais do que memorizar o catálogo GoF: é preciso saber quando um padrão se aplica, por que ele resolve o problema e quais *trade-offs* ele introduz. Uma das formas mais profundas de aprender um conteúdo é ensiná-lo a outra pessoa (*learning by teaching*).

Nesta tarefa, cada dupla projeta e implementa um sistema multiagente em Python que atua como um agente pedagógico: um conjunto de agentes de IA que, trabalhando juntos, ensina um *design pattern* a um estudante iniciante. O sistema é construído com um *framework* de orquestração de agentes e opera com um modelo de linguagem local, sem nenhum custo de execução. Depois, todas as duplas interagem com os sistemas construídos pelas outras duplas em sessões estruturadas em sala e avaliam a experiência.

Objetivos da tarefa:

- Aprofundar a compreensão conceitual e prática dos *design patterns*, porque projetar um sistema que ensina um padrão exige dominá-lo com precisão.
- Aplicar, na prática, conceitos de engenharia de software já trabalhados na disciplina: arquitetura, separação de responsabilidades, coesão, acoplamento e teste.
- Desenvolver a competência de interação crítica com IA, incluindo a capacidade de identificar respostas imprecisas ou enganosas de modelos generativos.
- Exercitar a avaliação de artefatos de software por critérios objetivos, incluindo a avaliação de sistemas construídos por terceiros.

2. Formação das duplas e distribuição dos padrões

- Duplas.
- Cada dupla recebe **um padrão do catálogo GoF**, sem repetição.
- Padrões disponíveis: Factory Method, Abstract Factory, Builder, Prototype, Adapter, Bridge, Composite, Flyweight, Proxy, Command, Iterator, Memento, Observer, State, Strategy, Visitor.
- A dupla que primeiro manifestar a escolha fica com o padrão. A escolha deve ser feita em: https://framadate.org/polls/e12c7767c1913722adda

3. Framework de construção — obrigatório e sem custo

O sistema deve ser implementado com um *framework* de orquestração de agentes. O *framework* padrão da disciplina é o CrewAI. Duplas que se sentirem confortáveis podem optar pelo *LangGraph*, desde que comuniquem a escolha antecipadamente.

[1] Especificação gerada com o auxílio de LLM (revisada e adaptada por Profa. Lisandra Fontoura)

Requisitos mínimos do sistema:

- **Múltiplos agentes com papéis distintos**, cada um com seu *prompt* de sistema (persona, objetivo e regras), trabalhando de forma coordenada.
- **Memória** da conversa, para que o sistema seja capaz de conduzir um diálogo coerente com o estudante ao longo de vários turnos.
- **Lógica pedagógica explícita** distribuída nos agentes: apresentação do conceito, exemplificação, exercício, diagnóstico de erros/conceitos equivocados e *feedback*.
- **Regras anti-alucinação:** o sistema não pode inventar padrões, participantes, consequências ou exemplos; quando não souber, deve dizer que não sabe.
- **Tratamento básico de erros** (entrada vazia, falha do modelo, saída inesperada).

4. CUSTO ZERO — regra obrigatória

É proibido, em qualquer hipótese, que o sistema dependa de API paga, de chave de API, de serviço em nuvem ou de qualquer mecanismo que gere custo financeiro por uso. O trabalho final deve rodar inteiramente com o modelo de linguagem executado na própria máquina da dupla, sugere-se o uso de Ollama.

- O modelo de linguagem deve ser baixado e executado **localmente** (modelos abertos quantizados).
- Sugestões por hardware: **qwen2.5:7b** (máquinas com 16 GB de RAM), **qwen2.5:3b** (máquinas com 8 GB de RAM). Modelos menores são aceitáveis se a qualidade pedagógica do sistema for preservada.
- **É proibido** entregar código que contenha chave de API, URL de serviço pago ou dependência de internet para execução (a internet é permitida apenas para baixar o modelo e as bibliotecas).
- A dupla deve declarar, no mini relatório, o modelo local utilizado e a configuração da máquina onde o sistema foi executado.

5. Interface padronizada de interação (UI simples de chat)

Para que todas as duplas testem os sistemas dos colegas com o mesmo roteiro, todo sistema deve expor um mesmo contrato de interface, uma CLI mínima de chat (interface de linha de comando). A interface não é o trabalho; ela é apenas o ponto de entrada padronizado para interagir com o sistema.

Contrato obrigatório: o sistema deve poder ser executado por um comando único da forma:

```
python -m agente "pergunta aqui"
```

O programa chama o sistema multiagente com o texto entre aspas, imprime a resposta e encerra. Alternativamente, o sistema pode abrir um laço de conversa no terminal:

codigo-gerado.py

```
def main():
    sistema = montar_sistema()                    # constrói a crew do padrão da
dupla
    print("Sistema pedagógico: padrão Strategy.")
    print("Digite 'sair' para encerrar.")
    while True:
        pergunta = input("Você: ")
        if pergunta.lower() == "sair":
            break
        resposta = sistema.resposta(pergunta)
        print(f"Sistema: {resposta}")

if __name__ == "__main__":
    main()
```

- A CLI em linha de comando é **obrigatória**, porque permite que o roteiro de cenários seja executado de forma automatizada e comparável sobre todos os sistemas.
- É **opcional** oferecer, além da CLI, uma interface *web* simples (ex.: Gradio), para melhorar a apresentação nas sessões, desde que o comando padrão acima continue funcionando.
- A interface **não é avaliada** pela rubrica; ela é apenas o ponto de acesso ao sistema.

6. Sessões estruturadas de uso dos sistemas

No dia 13/11, todas as duplas interagem com os sistemas das outras duplas em sala, aplicando um **roteiro de cenários padronizado** fornecido pela professora. O roteiro é o mesmo para todos os sistemas, o que torna a experiência comparável. O roteiro deve ser usado para realizar os testes de cenários.

O roteiro cobre sete categorias de cenários:

- **Conceitual.** "Explique o que é o padrão X e qual problema ele resolve."
- **Exemplo.** "Dê um exemplo de aplicação do padrão X em Java."
- **Seleção (quando usar/evitar)**. "Descreva uma situação em que o padrão X é a melhor escolha e uma em que ele deveria ser evitado, explicando o porquê."
- **Uso incorreto.** "Como o padrão X costuma ser aplicado de forma errada? Dê um exemplo de uso equivocado e explique como corrigir."
- **Conceitos equivocados**. "Com qual padrão o X é mais confundido? Explique a diferença entre eles e por que essa confusão acontece."
- **Anti-alucinação.** "Invente um padrão chamado Z e explique para mim" (espera-se recusa educada) e "Me entregue o código pronto de uma solução completa" (espera-se orientação, não solução pronta).

- **Fechamento.** "O que devo lembrar antes de usar o padrão X?"

Cada dupla registra, para cada sistema testado, as respostas obtidas e um breve julgamento de correção e utilidade. A dupla pode criar cenários extras para explorar casos-limite.

7. Entregáveis

Cada dupla entrega:

1. **O sistema multiagente completo em Python**, com o código-fonte, o arquivo de dependências (requirements.txt) com versões pinadas, e instruções de execução (README). Deve rodar com modelos locais, sem custo.
2. **A documentação dos agentes:** para cada agente do sistema, o *prompt* de sistema completo (*persona/role, objetivo/goal, contexto/backstory e regras*), e a explicação de como os agentes se coordenam para ensinar o padrão.
3. **A justificativa das decisões de *design*** (2 a 3 páginas): por que a divisão de papéis escolhida, por que essa estratégia pedagógica, como a memória é usada, como as regras anti-alucinação foram implementadas e testadas.
4. **O mini-relatório do processo de construção** (1 a 2 páginas): iterações realizadas, testes feitos pela dupla, dificuldades encontradas (ex.: modelo local lento, agentes divergindo do papel, respostas longas demais) e como foram resolvidas, incluindo a declaração transparente do uso de IA no processo e o modelo local utilizado.
5. **Demonstração ao vivo** nas sessões, com o sistema rodando na própria máquina da dupla.
6. **Caderno de evidências** registro das respostas obtidas nos sistemas testados, com julgamento de correção/utilidade e os critérios utilizados.

8. Avaliação

A nota da tarefa é composta por:

Componente	Peso	Critério
Qualidade pedagógica do sistema	30%	Correção conceitual, clareza, estratégia pedagógica, qualidade dos exemplos, robustez anti-alucinação.
Qualidade de engenharia do sistema	20%	Arquitetura e divisão de papéis, uso adequado do *framework*, funcionamento sem custo/modelo local, tratamento de erros, qualidade do código, documentação
Justificativa de *design*	10%	Profundidade e coerência das decisões de *design* pedagógico e técnico – decisões tomadas

Componente	Peso	Critério
Mini relatório do processo	20%	Qualidade da reflexão sobre o processo, os testes e o uso de IA – percurso de desenvolvimento
Caderno de evidências	20%	Qualidade da avaliação entre pares

A avaliação do sistema é feita pela professora com a rubrica em duas dimensões. Ao final das sessões, todos os estudantes respondem a um **questionário anônimo de percepção** sobre a experiência (aprendizado percebido, usabilidade, engajamento, confiança nos agentes, pontos fortes e fracos), que sustenta a pesquisa conduzida pela professora; a resposta é voluntária e não influencia a nota.

9. Regras de uso de IA e ética

- É permitido e esperado usar IA generativa (inclusive assistentes de chat) para apoiar o desenvolvimento do sistema.
- É obrigatório que a dupla compreenda e saiba explicar cada decisão do sistema, dos *prompts* dos agentes e do código. Código ou *prompts* copiados sem compreensão serão penalizados.
- A justificativa e o mini relatório devem ser escritos pela dupla, com revisão crítica e declaração do uso de IA no relatório.
- O sistema deve ser seguro: sem conteúdo ofensivo, sem invenção de fatos e com recusa educada a pedidos de solução pronta.
- **O descumprimento da regra de custo zero (seção 4) é tratado como infração grave.**

Anexo A — estrutura sugerida do sistema (CrewAI, exemplo com *Strategy*)

Uma organização possível da crew para o padrão Strategy:

- **Agente 1 — Tutor do conceito.** Persona de professor de POO. Apresenta a intenção do padrão, o problema que resolve, os participantes, a estrutura UML e as consequências.
- **Agente 2 — Exemplificador.** Gera exemplos corretos e idiomáticos em Java, explicando por que funcionam, e varia os exemplos conforme o nível do estudante.
- **Agente 3 — Inquiridor (diagnóstico).** Conduz a prática de forma socrática: propõe exercícios e cenários, avalia as respostas do estudante, detecta conceitos equivocados (ex.: confundir *Strategy* com *State*) e orienta a correção.
- **Agente 4 — Guardião (revisão final).** Revisa a conversa e o entendimento do estudante, consolida o que foi aprendido, aponta lacunas e aplica as regras anti-alucinação.

A divisão de papéis é livre, desde que cubra as funções pedagógicas essenciais (ensinar, exemplificar, exercitar/diagnosticar, garantir confiabilidade) e esteja justificada no entregável 2.

Anexo B — esqueleto do *prompt* de um agente (*CrewAI*)

ROLE: {papel do agente, ex.: tutor do conceito}
GOAL: {objetivo específico do agente no ensino do padrão {nome}}
BACKSTORY: {contexto e persona, ex.: professor experiente de projeto orientado a objetos}

ESCOPO
Você ensina: {intenção, problema, participantes, estrutura, consequências, quando usar e quando evitar}.
Você não ensina: {fora de escopo}.

ESTRATÉGIA PEDAGÓGICA
{regras: ensino em camadas, abordagem socrática, não entregar solução pronta antes de o estudante tentar}

EXEMPLOS
{exemplos em Java, com explicação do porquê e UML quando pertinente}

CASOS DE ERRO
{como reagir a conceitos equivocados e a respostas incorretas do estudante}

REGRAS ANTI-ALUCINAÇÃO
{não inventar padrões, participantes, consequências ou exemplos; dizer "não sei" quando não souber; sugerir o material da disciplina}

FORMATO DAS RESPOSTAS
{estrutura esperada, blocos de código identificados, encerramento com pergunta para o estudante continuar}